

Mathematics for Computer Science – 1

王立斌

School of Computer Science, South China Normal University

May 28, 2026

Table of contents

- 1 Introduction.
- 2 Numbers in Computer Systems.
- 3 Multiplication
- 4 Division.
- 5 The Euclidean Algorithm
- 6 The Extended Euclidean Algorithm

What is Mathematics for Computer Science – 1?

According to LLM17-MCS:

- Proof techniques – 证明方法
- Number Theory – 初等数论
- Abstract Algebra – 抽象代数
- Probability – 概率论
- Algorithms Analysis – 算法分析

What is Mathematics for Computer Science – 2?

Accoring to 《Concrete Mathematics》 (GKP93– 《具体数学》):

- Number Theory – 初等数论
- Probability – 概率论
- Algorithms Analysis – 算法分析相关的数学

What is Mathematics for Computer Science – 2?

According to 《Concrete Mathematics》 (GKP93– 《具体数学》):

- Number Theory – 初等数论
- Probability – 概率论
- Algorithms Analysis – 算法分析相关的数学

本人浅见!

《计算机科学中的数学》就是《离散数学》。

从 2024 年起,《计算机科学中的数学》第一次出现在 SCNU-CS, 以下专题为本课程的主线:

- Number Theory – 初等数论
- Abstract Algebra – 抽象代数
- Algorithms – 算法

从 2024 年起,《计算机科学中的数学》第一次出现在 SCNU-CS, 以下专题为本课程的主线:

- Number Theory – 初等数论
- Abstract Algebra – 抽象代数
- Algorithms – 算法

Notice!

强调, 我们关注编程与算法, 即所谓的“computational(计算性)”, 且以上专题将融合在一起。

- A Concrete Introduction to Number Theory and Algebra(CINTA, 本课程教材)
- Number Theory: A Friendly Introduction to Number Theory (FINT)
- Abstract Algebra: Abstract Algebra: Theory and Applications (AATA)
- Algorithms: Introduction to Algorithms (CLRS)

- A Concrete Introduction to Number Theory and Algebra(CINTA, 本课程教材)
- Number Theory: A Friendly Introduction to Number Theory (FINT)
- Abstract Algebra: Abstract Algebra: Theory and Applications (AATA)
- Algorithms: Introduction to Algorithms (CLRS)

Wanted!

Reading, thinking and programming.

本课程期望的目标.

同学们可以建立起最基本、正常的数学学习思维，并多做练习.

本课程期望的目标.

同学们可以建立起最基本、正常的数学学习思维，并多做练习.

本课程一种重要的思路.

Finding patterns. (寻找模式.)

AI 时代的数学课有什么特征.

- 更丰富的知识；更丰富的阅读资料；
- 更唾手可得的答案；

AI 时代的数学课有什么特征.

- 更丰富的知识；更丰富的阅读资料；
- 更唾手可得的答案；

AI 时代的数学课需要警惕什么？

- 知识、资料过多，学习路径的选择更复杂；
- 容易得到答案，让本来就缺乏思考的大学生更难以抗拒惰性，“不思考”的状况进一步恶化；

Number systems

$$\mathbb{N} = \{0, 1, 2, \dots\}$$

$$\mathbb{Z} = \{0, \pm 1, \pm 2, \pm 3, \dots\}$$

$$\mathbb{Q}$$

$$\mathbb{R}$$

Number systems

$$\mathbb{N} = \{0, 1, 2, \dots\}$$

$$\mathbb{Z} = \{0, \pm 1, \pm 2, \pm 3, \dots\}$$

$$\mathbb{Q}$$

$$\mathbb{R}$$

问题.

计算机系统数学结构主要是什么？

Simple Code:

```
1 unsigned char a = 255, b = 2;  
2 a = a + b;  
3 printf("a == %d \n", a);
```

问题.

这段程序输出什么？

Warm-up-2.

Simple Code:

```
1 int n = 32; /*try different values here;*/
2 unsigned int val = 1; /*or try a different type here;*/
3 for(int i = 1; i <= n; i++)
4 {
5     val = val * 2;
6     printf("What you get is %u \n.", val);
7 }
```

问题.

这段程序输出什么？

CSers 的计算方法.

请计算以下等式:

$$1 + 2 + 2^2 + 2^3 + \dots + 2^{10}$$

问题.

这道题的意义在哪里?

Observations:

- All computations in computers use fixed-size representations(定长计算).
- Addition (加法) is natural while multiplication (乘法) is not.
- Subtraction (减法) is the same as addition; the relation between multiplication and division (除法) is more complicated.

Observations:

- All computations in computers use fixed-size representations(定长计算).
- Addition (加法) is natural while multiplication (乘法) is not.
- Subtraction (减法) is the same as addition; the relation between multiplication and division (除法) is more complicated.

注意!

以下我们重点关注计算机系统中的加、减、乘、除.

Simple rules for addition and multiplication.

Well-known and useful rules of arithmetic:

- The last bit of an even(odd) number is 0(1);
- To multiply(divide) an unsigned integer by 2 is equivalent to shifting left(right) the number by 1 bit.

诡异的加法.

整数加法

```
1 //输入: 两个整数a和b
2 //输出: a与b的和
3 int add(int a, int b) {
4     if (b == 0) return a;
5     return add(a ^ b, (b & a) << 1);
6 }
```

问题.

如何理解这个算法？

要理解这个加法算法只需要正确回答出以下三个问题：

- ① $a \wedge b$ 得到的是什么？
- ② $(b \& a) \ll 1$ 得到的是什么？
- ③ 该算法为什么会终止？

学习目标.

本节的主要内容：朴素乘法、简单乘法（递归版本与迭代版本）。
建立算法分析的基本思路。

Naïve Multiplication.

阅读并思考以下代码，理解它的计算方法。

```
1  # Input: two integers a and b, where  $a \geq b$ .  
2  # Output: the product of a and b.  
3  def naive_multiply(a, b):  
4      if b == 0:  
5          return 0  
6      return a + naive_multiply(a, b - 1)
```

问题.

以上算法是否正确？效率如何？

Three key issues.

In general, three key issues should be considered when we encounter a new algorithm.

- **Correctness(正确性)**. Does the algorithm correctly achieve the goal?
- **Efficiency(效率、复杂度)**. How long does the algorithm take when it terminates?
- **Optimization(优化)**. Can we provide a better method?

Rule of Multiplication.

为了得到计算 $a * b$ 的优化算法，先思考以下递归的计算过程。

Rule of Multiplication.

$$a \cdot b = \begin{cases} 2(a \cdot \lfloor b/2 \rfloor) & \text{if } b \text{ is even;} \\ a + 2(a \cdot \lfloor b/2 \rfloor) & \text{if } b \text{ is odd.} \end{cases} \quad (1)$$

Simple multiplication 的伪代码.

```
1  # Input: two integers a and b.  
2  # Output: the product of a and b.  
3  def multiply(a, b):  
4      if b == 0:  
5          return 0  
6      if is_even(b): # the last bit of b is 0;  
7          return 2*multiply(a, b//2)  
8      else: # b is odd  
9          return 2*multiply(a, b//2) + a
```

问题.

以上算法是否正确？效率是否提高？

如何得到一种迭代式的乘法算法？考虑以下表达式：

Intuition.

For two n -bits integers a and b , view b as a binary number in position notation(位置记数法).

$$\begin{aligned} a \cdot b &= a \cdot (b_{n-1}2^{n-1} + b_{n-2}2^{n-2} + \cdots + b_12 + b_0) \\ &= b_{n-1}(a \cdot 2^{n-1}) + b_{n-2}(a \cdot 2^{n-2}) + \cdots + b_1(a \cdot 2) + b_0a \end{aligned}$$

Observations: 1. Every term in the last equation has $2^i a$, which means we must continue to do $a* = 2$

2. The bit b_i in every term plays as a flag, when $b_i = 1$, we must have $res+ = a$;

位置计数法给我们的启发:

观察.

- Every term in the last equation has $2^i a$, which means we must continue to do $a* = 2$
- The bit b_i in every term plays as a flag, when $b_i = 1$, we must have $res+ = a$;

Example.

To compute 3×10 .

Example.

To compute 3×10 .

10's binary string is 1010.

Example.

To compute 3×10 .

10's binary string is 1010.

Repeatedly multiply 3 by 2^i and get: 3, 6, 12, 24.

Example.

To compute 3×10 .

10's binary string is 1010.

Repeatedly multiply 3 by 2^i and get: 3, 6, 12, 24.

Choose the right terms to add, they are 6 and 24.

Example.

To compute 3×10 .

10's binary string is 1010.

Repeatedly multiply 3 by 2^i and get: 3, 6, 12, 24.

Choose the right terms to add, they are 6 and 24.

Return the result which is 30.

Simple multiplication.

Analysis

- Correctness. Obviously...
- Efficiency. $O(n^2)$ bit operations. Why?
- Optimization. To be or not to be...

Notice!

The multiplication is implemented NOT using multiplication !

思考.

请不要把这里的“加、减、乘、除”仅仅视为计算机系统中需要实现的计算方法，而应该把它们理解为某种“模式”、某种“思维方式”。请在后续课程中不断体会。

练习题.

对比以下两种数列求积算法，说明并验证它们的效率。

```
1 def product1(X):  
2     res = 1  
3     for i in range(len(X)):  
4         res *= X[i]  
5     return res
```

```
1 def product2(X):  
2     if len(X) == 0: return 1  
3     if len(X) == 1: return X[0]  
4     if len(X) == 2: return X[0] * X[1]  
5     l = len(X)  
6     return product2(X[0:(l + 1) // 2]) * product2(X[(l + 1) // 2: l])
```

本节学习目标.

本节的主要内容：除法算法、良序公理、除法算法的证明。

Theorem

(Division Algorithm) Given integers a and b , with $b > 0$, there exist unique integers q and r , called quotient (商) and remainder (余数) respectively, such that

$$a = qb + r$$

where $0 \leq r < b$.

Remark.

除法算法并不仅仅是算法或定理。

Geometric interpretation of division.

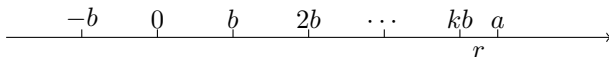


Figure: Geometric interpretation of division.

Remark.

除法算法的几何解释能在后续帮助我们设计算法，请留意。

在证明除法算法之前，先解释一种常用公理。

Principle of Well-Ordering(良序原则).

Every nonempty subset of natural numbers contain a least element.

Proof of division theorem.

The proof contains two parts: existence(存在性) and uniqueness(唯一性).

Proof.

- **Existence of q and r .** 证明存在性使用什么方法？

Proof of division theorem.

The proof contains two parts: existence(存在性) and uniqueness(唯一性).

Proof.

- **Existence of q and r .** 证明存在性使用什么方法？

构造法：Given a and b , construct a set

$$S = \{a - bk : k \in \mathbb{Z} \text{ and } (a - bk) \geq 0\}.$$



Proof.

- It is easy to show that S is nonempty. By the **Principle of Well-Ordering**, there is a smallest member $r \in S$, and $r = a - qb$. Therefore, $a = qb + r$, $r \geq 0$. To show $r < b$ is left as an exercise.
- **Uniqueness of q and r .** This is an exercise. [提示：证明唯一性使用什么方法？反证法！]



算法的实现：

```
1 def naive_divide(a, b):  
2     q, r = 0, 0  
3     while(a >= b):  
4         a, q = a - b, q + 1  
5     r = a  
6     return q, r
```

问题.

算法是否正确？效率如何？能否优化？

Simple Division.

```
1 def divide(a, b):  
2     if(a == 0):  
3         return 0, 0  
4     q, r = divide(a // 2, b)  
5     q, r = 2 * q, 2 * r  
6     if(a & 1): #if a is an odd number  
7         r = r + 1  
8     if(r >= b):  
9         r, q = r - b, q + 1  
10    return q, r
```

问题.

算法是否正确？效率如何？是否对朴素除法的优化？为什么？

重要提示.

Multiplication can be implemented using only addition and shifts, while division can be implemented using only subtraction and shifts. More importantly, the point is not merely implementation, but a way of thinking that will reappear throughout the course.

以下我们转入一个新的主题：欧几里得算法。先介绍一些定义与概念。

- **Divisibility.** Let a and b be integers, a is said to be *divisible* by b when $a = qb$ for some integer q , we write $b \mid a$.
- **Common divisor.** An integer d is called a common divisor of a and b if $d \mid a$ and $d \mid b$.
- **Greatest common divisor.** An integer d is the greatest common divisor(gcd) of a and b , if d is a common divisor of a and b , and for any other common divisor of a and b , says d' , then $d' \mid d$.
- **Relatively prime.** If $\gcd(a, b) = 1$, we say that a and b are relatively prime.

The Euclidean Algorithm.

Euclidean algorithm(gcd algorithm).

Given two positive integers a and b with $a \geq b$:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

where $a \bmod b$ means to divide a by b to get the remainder r .

一个计算实例。

Example

Given $a = 12345$ and $b = 678$,

$$\begin{aligned}\gcd(12345, 678) &= \gcd(678, 141) \\ &= \gcd(141, 114) \\ &= \gcd(114, 27) \\ &= \gcd(27, 6) \\ &= \gcd(6, 3) \\ &= \gcd(3, 0)\end{aligned}$$

Hence, $\gcd(12345, 678) = 3$.

The Euclidean Algorithm.

gcd 算法的递归实现如下：

```
1  # Input: positive integers a and b, with  $a > b$  .  
2  # Output: the greatest common divisor of a and b.  
3  def gcd(a, b):  
4      if (b == 0):  
5          return a  
6      else:  
7          return gcd(b, a % b)
```

Correctness of The Euclidean Algorithm.

Correctness. It is enough to show a slightly simpler rule:

$$\gcd(a, b) = \gcd(a - b, b).$$

From this rule, we can repeatedly subtract b from a (recall the Naive division algorithm) and derive the final rule as follows:

$$\begin{aligned}\gcd(a, b) &= \gcd(a - b, b) \\ &= \gcd((a - b) - b, b) \\ &= \dots \\ &= \gcd(a \bmod b, b).\end{aligned}$$

Correctness of The Euclidean Algorithm.

To show $\gcd(a, b) = \gcd(a - b, b)$, we have two directions to prove.

- any integer that divides both a and b must also divides $a - b$, thus $\gcd(a, b) \leq \gcd(a - b, b)$.
- any integer that divides both $a - b$ and b must also divides a and b , thus $\gcd(a - b, b) \leq \gcd(a, b)$.

Efficiency of The Euclidean Algorithm.

Efficiency. To understand that the gcd algorithm is quite fast, we should be aware of the fact that $a \bmod b$ is a substantial reduction. It is easy to check:

Lemma

If $a \geq b$, then $(a \bmod b) < a/2$.

Hence, after two consecutive rounds, both a and b are at least halved in value, means the algorithm will terminate after $\mathcal{O}(\log a)$ recursive calls.

Optimization of The Euclidean Algorithm.

Can we do better? The *binary gcd algorithm*, also known as Stein's algorithm, has the same function as the gcd algorithm. This algorithm avoids complex operations, such as division and multiplication; instead, it relies only on subtraction, and more efficient bit right shift and bit left shift operations. The analysis of binary gcd algorithm is an exercise.

Theorem

(Bézout) Let a and b be nonzero integers. Then there exist integers r and s such that

$$\gcd(a, b) = ar + bs$$

Furthermore, the greatest common divisor of a and b is unique.

Theorem

(Bézout) Let a and b be nonzero integers. Then there exist integers r and s such that

$$\gcd(a, b) = ar + bs$$

Furthermore, the greatest common divisor of a and b is unique.

Proof.

构造集合

$$S = \{am + bn : m, n \in \mathbb{Z} \text{ 且 } am + bn > 0\}.$$

显然，集合 S 非空，根据良序原则，取其中最小值 $d = ar + bs$ 。然后证明两个属性： d 是 a 和 b 的公因子（提示：除法算法!）；如果存在 a 和 b 的公因子 d' ，则 $d' \mid d$ 。这样就证明了 $d = \gcd(a, b)$ 。 □

Proof.

证明 d 整除 a 。根据除法算法，记 $a = dq + r'$ ， $0 \leq r' < d$ 。证明 $r' = 0$ 。如果 $r' > 0$ ，则有：

$$\begin{aligned}r' &= a - dq \\&= a - (ar + bs)q \\&= a(1 - qr) + b(-qs)\end{aligned}$$

所以， r' 也落在集合 S 中，但是，这与 d 是集合 S 中最小元素矛盾。因此， r' 只能是 0。同理，可证 d 整除 b 。请读者完成余下细节，留作练习。 □

Bézout 定理的证明还显示这样一个重要的事实:

$\gcd(a, b)$ 等于 a 和 b 所有线性组合值的最小值。

记住这个事实, 它可经常帮助我们解题。比如, 我们立即可得以下推论。

Corollary

整数 a 和 b 互素当且仅当存在整数 r 和 s 使得 $ar + bs = 1$ 。

Theorem

设 $a, b \in \mathbb{Z}$, p 为素数。如果 $p \mid ab$, 则 $p \mid a$ 或 $p \mid b$ 。

Bézout's Theorem 的应用.

Theorem

设 $a, b \in \mathbb{Z}$, p 为素数。如果 $p \mid ab$, 则 $p \mid a$ 或 $p \mid b$ 。

Proof.

假设 $p \nmid a$, 证明 $p \mid b$ 。因为 $p \nmid a$ 且 p 是素数, 则 $\gcd(a, p) = 1$, 即存在 $r, s \in \mathbb{Z}$ 使得 $ar + ps = 1$ 。因此,

$$b = b(ar + ps) = abr + pbs.$$

既然 $p \mid ab$, 那么 p 同时整除 abr 和 pbs , 则 p 必然整除 b . $\square$

Corollary

设 $a, b, n \in \mathbb{Z}$, $n > 0$ 。如果 $n \mid ab$ 且 $\gcd(a, n) = 1$, 则 $n \mid b$.

Proof.

使用以上类似的方法, 易得, 留作练习。



The Extended Euclidean Algorithm.

The extended Euclidean algorithm.

The *extended Euclidean algorithm* (egcd-algorithm) is a simple extension of the Euclidean algorithm. Given two nonzero integers a and b , it efficiently computes the *Bézout coefficients* r and s , and plays an important role in modular arithmetic.

The Extended Euclidean Algorithm.

Example

Given $a = 19$, $b = 13$, to find r and s , s.t. $ar + bs = \gcd(19, 13)$.

$$a \cdot 1 + b \cdot 0 = 19 \quad (2)$$

$$a \cdot 0 + b \cdot 1 = 13 \quad (3)$$

$$a \cdot 1 + b \cdot (-1) = 6 \quad (4)$$

$$a \cdot (-2) + b \cdot 3 = 1 \quad (5)$$

Finally, we have $r = -2$, $s = 3$ and $d = 1$, which is the answer.

Recapitulate the process.

- Write out two obviously true linear equations.;
- Repeatedly subtract some integer multiple of one equation from the other, until the right-hand side of one equation achieves $\gcd(a, b)$.

Two Observations.

At least two facts should be noticed

- The right-hand sides of these equations undergo the same iterations as in the Euclidean algorithm.
- The a and b are just place holders, they are not involved in the computation, we can safely get rid of them in the equations.

例子

Given $a = 13$, $b = 9$, to find r and s , s.t. $ar + bs = \gcd(13, 9)$. We write out a matrix:

$$\begin{pmatrix} 1 & 0 & 13 \\ 0 & 1 & 9 \end{pmatrix}$$

If we subtract row 2 from row 1, we get a new matrix:

$$\begin{pmatrix} 0 & 1 & 9 \\ 1 & -1 & 4 \end{pmatrix}$$

例子-续

Then we subtract 2 multiple of row 2 from row 1, we get:

$$\begin{pmatrix} 1 & -1 & 4 \\ -2 & 3 & 1 \end{pmatrix}$$

Finally, we have $r = -2$, $s = 3$ and $d = 1$, which is the answer.

The Extended Euclidean Algorithm.

From the perspective of matrix, the process starts with a matrix

$$M = \begin{pmatrix} 1 & 0 & a \\ 0 & 1 & b \end{pmatrix}$$

which satisfies

$$M \begin{pmatrix} a \\ b \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

By repeatedly subtracting a multiple of one row of M from the other row or exchanging the two rows, finally we obtain a matrix M'

$$M' = \begin{pmatrix} r & s & d' \\ R & S & 0 \end{pmatrix}$$

The Extended Euclidean Algorithm.

$$M' = \begin{pmatrix} r & s & d \\ R & S & 0 \end{pmatrix}$$

satisfies

$$M' \begin{pmatrix} a \\ b \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

That means $ar + bs = d$.

课堂练习.

Given $a = 252$, $b = 198$, to find r , s and $d = \gcd(a, b)$, s.t.
 $ar + bs = d$.

The Extended Euclidean Algorithm.

The extended Euclidean algorithm is essentially the process to calculate Bézout coefficients we described above.

```
1  # Input: integers a and b, with a > b .
2  # Output: d=gcd(a,b), and
3  # r and s s.t. d = a*r + b*s
4  def egcd(a, b):
5      r0, r1, s0, s1 = 1, 0, 0, 1
6      while(b):
7          q, a, b = a//b, b, a%b
8          r0, r1 = r1, r0 - q*r1
9          s0, s1 = s1, s0 - q*s1
10     return a, r0, s0
```

Analysis of The Extended Euclidean Algorithm.

- Correctness: it is easy.
- Efficiency: it is trivial.
- Optimization: binary-egcd.

Concluding Remarks.

- The content is easy, but is not trivial.
- Algorithms should be focused by CSers.
- Useful tools should be mastered.

Happy ending.

- Any questions?
- Feedback is welcome: lbwang@gmail.com.
- Thanks!